

Gerador automático de relatório de vendas

Uma atividade prática para integrar pandas, openpyxl e automação de planilhas em um fluxo empresarial completo.

Duração	Formato	Entrega
2 encontros	Individual ou em grupo	Programa + relatório Excel

Objetivo

Criar um programa capaz de localizar planilhas de vendas, consolidar os registros, tratar os dados, calcular indicadores e gerar um arquivo Excel final já formatado e pronto para análise.

Ideia central

A automação deve funcionar novamente quando novos arquivos forem adicionados. O relatório final não deverá depender de correções manuais.

Adicionar arquivos -> Executar o programa -> Receber o relatório pronto

Cenário do projeto

Uma empresa registra suas vendas em arquivos separados por mês. O trabalho atual é repetitivo: abrir cada arquivo, copiar dados, calcular totais e montar resumos.

ENTRADA	PROCESSAMENTO	SAÍDA
vendas_janeiro.xlsx vendas_fevereiro.xlsx vendas_marco.xlsx	Ler, juntar, limpar, calcular e resumir	relatorio_final.xlsx com várias abas

Estrutura das planilhas de entrada

Data	Vendedor	Produto	Categoria	Quantidade	Valor Unitário
10/01/2026	Ana	Notebook	Informática	2	R\$ 3.500,00

Problema atual

- ✓ Os arquivos estão separados por mês.
- ✓ Há risco de registros duplicados ou incompletos.
- ✓ Os totais precisam ser recalculados a cada atualização.
- ✓ A apresentação do relatório é feita manualmente.

Solução esperada

- ✓ Um único programa processa todos os arquivos.
- ✓ O pandas prepara e analisa os dados.
- ✓ O openpyxl organiza a apresentação final.
- ✓ O processo pode ser repetido sempre que necessário.

Regra de automação

Nenhum resultado importante deverá depender de copiar, colar ou editar manualmente o arquivo final.

O papel de cada biblioteca

As duas bibliotecas trabalham juntas, mas resolvem partes diferentes do problema.

pandas - conteúdo e análise

- Ler arquivos Excel
- Juntar tabelas com concat
- Tratar ausências e duplicidades
- Padronizar textos, datas e números
- Criar a coluna Total
- Filtrar, agrupar e resumir
- Exportar diferentes tabelas

openpyxl - apresentação do Excel

- Formatar cabeçalhos
- Ajustar larguras das colunas
- Aplicar formatos de moeda e data
- Congelar o cabeçalho
- Adicionar filtros
- Destacar valores
- Criar gráficos no arquivo final

Fluxo obrigatório

1	Localizar	Encontrar automaticamente os arquivos .xlsx da pasta.
2	Ler	Transformar cada planilha em um DataFrame.
3	Tratar	Corrigir tipos, ausências, textos e duplicidades.
4	Analisar	Criar totais, filtros, agrupamentos e indicadores.
5	Exportar	Gerar um Excel com várias abas.
6	Formatar	Aplicar acabamento profissional com openpyxl.

Pergunta orientadora

Se uma nova planilha mensal for colocada na pasta, o programa conseguirá atualizar o relatório sem alterações no código?

Requisitos mínimos

O projeto será considerado funcional quando cumprir todos os itens abaixo.

- ✓ Ler pelo menos três planilhas de entrada.
- ✓ Juntar os registros automaticamente em um único DataFrame.
- ✓ Remover duplicidades e tratar valores ausentes com uma regra definida.
- ✓ Criar a coluna Total = Quantidade x Valor Unitário.
- ✓ Gerar pelo menos dois resumos, como vendedor e produto.
- ✓ Criar um arquivo Excel com diferentes abas.
- ✓ Formatar cabeçalhos, moedas, datas e larguras de colunas.
- ✓ Salvar o relatório final e exibir uma mensagem de conclusão.

Ponto de partida sugerido

```
from pathlib import Path
import pandas as pd

pasta = Path("planilhas_vendas")
tabelas = []

for arquivo in pasta.glob("*.xlsx"):
    tabela = pd.read_excel(arquivo)
    tabela["Arquivo de Origem"] = arquivo.name
    tabelas.append(tabela)

dados = pd.concat(tabelas, ignore_index=True)
dados["Total"] = dados["Quantidade"] * dados["Valor Unitário"]
```

Atenção ao teste

O programa deve informar claramente quando a pasta estiver vazia, quando um arquivo estiver aberto no Excel ou quando faltar uma coluna obrigatória.

Boa prática

A coluna Arquivo de Origem facilita auditoria: se um dado estiver incorreto, será possível descobrir de qual planilha ele veio.

Como deverá ser o relatório final

O arquivo relatorio_final.xlsx deverá reunir dados detalhados e informações resumidas.

Aba	Conteúdo esperado
Dados Consolidados	Todos os registros tratados, incluindo Total e Arquivo de Origem.
Resumo por Vendedor	Total vendido e quantidade de vendas de cada vendedor.
Resumo por Produto	Quantidade vendida e faturamento de cada produto.
Inconsistências	Linhas com informações ausentes ou inválidas, quando existirem.

Exportação com pandas

```
with pd.ExcelWriter("relatorio_final.xlsx", engine="openpyxl") as escritor:  
    dados.to_excel(escritor, sheet_name="Dados Consolidados", index=False)  
    resumo_vendedores.to_excel(  
        escritor, sheet_name="Resumo por Vendedor", index=False  
    )  
    resumo_produtos.to_excel(  
        escritor, sheet_name="Resumo por Produto", index=False  
    )
```

Acabamento com openpyxl

- ✓ Cabeçalho com fonte em negrito e cor de preenchimento.
- ✓ Larguras ajustadas para evitar textos cortados.
- ✓ Moedas no padrão R\$ e datas em dia/mês/ano.
- ✓ Cabeçalho congelado e filtros ativados.
- ✓ Gráfico de barras com o total vendido por vendedor.

Tarefa obrigatória: gráfico de vendas

Além das tabelas, o relatório deverá apresentar um gráfico que facilite a comparação do desempenho dos vendedores.

Objetivo da tarefa

Criar um gráfico de barras na aba Resumo por Vendedor, usando os nomes como categorias e o total vendido como valor.

Resultado esperado

Ao executar novamente o programa, o gráfico deverá ser recriado com os valores atualizados, sem ajustes manuais no Excel.

Código-base com openpyxl

```
from openpyxl.chart import BarChart, Reference

aba_resumo = planilha["Resumo por Vendedor"]

grafico = BarChart()
grafico.title = "Total vendido por vendedor"
grafico.y_axis.title = "Valor vendido (R$)"
grafico.x_axis.title = "Vendedor"

valores = Reference(
    aba_resumo, min_col=2, min_row=1, max_row=aba_resumo.max_row
)
nomes = Reference(
    aba_resumo, min_col=1, min_row=2, max_row=aba_resumo.max_row
)

grafico.add_data(valores, titles_from_data=True)
grafico.set_categories(nomes)
grafico.height = 8
grafico.width = 14

aba_resumo.add_chart(grafico, "D2")
planilha.save("relatorio_final.xlsx")
```

Requisitos do gráfico

- ✓ Usar os dados da aba Resumo por Vendedor.
- ✓ Apresentar título e títulos dos eixos.
- ✓ Exibir todos os vendedores existentes.
- ✓ Não cobrir a tabela com o gráfico.
- ✓ Atualizar ao executar o programa novamente.

Perguntas para responder

- ✓ Quem apresentou o maior total vendido?
- ✓ Qual é a diferença entre o primeiro e o segundo colocado?
- ✓ O maior total veio de mais vendas ou de vendas mais caras?
- ✓ Que outra visualização ajudaria nessa análise?

Roteiro para os dois encontros

Sugestão de divisão para encontros de aproximadamente 90 minutos.

ENCONTRO 1

Dados corretos primeiro

Objetivo: terminar com as tabelas tratadas e os resumos calculados.

- ✓ 10 min - Ler o cenário e dividir responsabilidades.
- ✓ 15 min - Organizar a pasta e conferir as colunas.
- ✓ 25 min - Ler e consolidar os arquivos.
- ✓ 20 min - Limpar e padronizar os dados.
- ✓ 15 min - Criar Total e os agrupamentos.
- ✓ 5 min - Executar um primeiro teste completo.

ENCONTRO 2

Relatório pronto depois

Objetivo: exportar, formatar, testar e apresentar o resultado.

- ✓ 15 min - Gerar as diferentes abas do Excel.
- ✓ 25 min - Aplicar a formatação com openpyxl.
- ✓ 15 min - Implementar e testar o gráfico de vendas.
- ✓ 15 min - Testar com arquivos novos e erros comuns.
- ✓ 10 min - Organizar a entrega e comentários no código.
- ✓ 10 min - Apresentar a solução para a turma.

Marcos de acompanhamento

Ao final do primeiro encontro, o relatório pode estar visualmente simples, mas os dados e cálculos devem estar corretos. No segundo encontro, o foco passa a ser apresentação, testes e confiabilidade.

Divisão sugerida para grupos

- ✓ Pessoa 1: leitura, consolidação e limpeza dos dados.
- ✓ Pessoa 2: cálculos, resumos e exportação das abas.
- ✓ Pessoa 3: formatação, testes e documentação.

O que entregar

A entrega deve permitir que outra pessoa execute o programa e compreenda o resultado.

Estrutura sugerida

```
projeto_relatorio/  
|-- planilhas_vendas/  
|   |-- vendas_janeiro.xlsx  
|   |-- vendas_fevereiro.xlsx  
|   |-- vendas_marco.xlsx  
|-- relatorio_vendas.py  
|-- relatorio_final.xlsx  
-- README.txt
```

Itens obrigatórios

- ✓ Código Python organizado e comentado.
- ✓ Planilhas usadas como entrada.
- ✓ Arquivo Excel final gerado pelo programa.
- ✓ README com instruções para executar.
- ✓ Breve apresentação do resultado.

Critérios de avaliação - 10 pontos

Critério	Pontos	O que será observado
Leitura e consolidação	2,0	Localização e união automática dos arquivos.
Tratamento dos dados	2,0	Tipos, ausências, duplicidades e padronização.
Cálculos e resumos	2,0	Total correto e indicadores relevantes.
Relatório Excel	2,0	Abas, formatação, filtros, gráfico e legibilidade.
Organização e testes	1,0	Código claro, mensagens e tratamento de erros.
Apresentação	1,0	Explicação das decisões e demonstração do programa.

Desafios extras

Os itens abaixo são opcionais e podem ser escolhidos pelos grupos que concluírem os requisitos mínimos.

Análise e visualização

- Segundo gráfico com o resumo mensal
- Gráfico por categoria
- Produto com maior faturamento
- Destaque para vendas acima de R\$ 5.000
- Comparação entre períodos

Experiência e confiabilidade

- Escolha de pasta com Tkinter
- Nome do relatório com a data atual
- Tratamento de exceções
- Registro de inconsistências
- Log simples da execução

Versão alternativa para contabilidade

A mesma estrutura pode ser aplicada a um relatório de receitas e despesas. Nesse caso, as colunas podem ser Data, Descrição, Categoria, Tipo, Valor e Centro de Custo. O programa deverá calcular receitas, despesas, saldo, gastos por categoria e movimentações inconsistentes.

Checklist antes da entrega

- ✓ O programa funciona sem editar manualmente o relatório final?
- ✓ Uma nova planilha pode ser adicionada sem modificar o código?
- ✓ Os totais foram conferidos com alguns cálculos manuais?
- ✓ Datas, moedas, cabeçalhos e colunas estão legíveis?
- ✓ Erros comuns apresentam mensagens compreensíveis?
- ✓ Outra pessoa consegue executar o projeto seguindo o README?

Perguntas para a apresentação

Qual parte foi automatizada? Quais problemas de dados foram encontrados? Como pandas e openpyxl dividiram as responsabilidades? O que aconteceria ao adicionar um novo mês?

O objetivo não é apenas criar uma planilha bonita. É construir um processo repetível, confiável e útil.